POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

A lightweight canonicalization engine in C

WILLIAM ZHANG
Département de Electrical Engineering

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
Electrical Engineering

July 2026

© William Zhang, 2026.

POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

Ce mémoire intitulé :

A lightweight canonicalization engine in C

présenté par **William ZHANG**
en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
a été dûment accepté par le jury d'examen constitué de :

Danielle DUBOIS, présidente
Sébastien VIGOT, membre et directeur de recherche
Marie COUTURE, membre et codirectrice de recherche
Jean TREMBLAY, membre
Joseph BROWN, membre externe

DEDICATION

To all the past, present and future CVXPY contributors

ACKNOWLEDGEMENTS

Texte / Text.

RÉSUMÉ

Le résumé est un bref exposé du sujet traité, des objectifs visés, des hypothèses émises, des méthodes expérimentales utilisées et de l'analyse des résultats obtenus. On y présente également les principales conclusions de la recherche ainsi que ses applications éventuelles. En général, un résumé ne dépasse pas trois pages.

Le résumé doit donner une idée exacte du contenu du mémoire ou de la thèse. Ce ne peut pas être une simple énumération des parties du manuscrit. Le but est de présenter de façon précise et concise la nature, l'envergure de la recherche, les sujets traités, les questions de recherche ou les hypothèses soulevées, les méthodes utilisées, les principaux résultats ainsi que les conclusions retenues. Un résumé ne doit jamais comporter de références ou de figures.

ABSTRACT

Written in English, the abstract is a brief summary similar to the previous section (Résumé). However, this section is not a word for word translation of the abstract in French.

The abstract is a brief statement of the subject matter, objectives, research questions or hypotheses, experimental methods and analysis of results. It also presents the main research conclusions and their possible applications. In general, an abstract should not exceed three pages.

The abstract should provide an exact idea of the thesis or dissertation's contents and it cannot be a simple enumeration of the manuscript's parts. The goal is to precisely and concisely present the nature and scope of the research. An abstract should never include references or figures. If the thesis or the dissertation is in English, the résumé (French-language abstract) should come first followed by the abstract.

TABLE OF CONTENTS

DEDICATION iii

ACKNOWLEDGEMENTS iv

RÉSUMÉ v

ABSTRACT vi

LIST OF TABLES ix

LIST OF FIGURES x

LIST OF SYMBOLS AND ACRONYMS xi

CHAPTER 1 INTRODUCTION 1

CHAPTER 2 LITERATURE REVIEW 4
2.1 Canonicalization 5
2.2 Matrix stuffing 9
2.3 Coefficient extraction 12
2.4 Computing derivatives 12
2.5 Sparse matrices 16

CHAPTER 3 A lightweight canonicalization engine 19
3.1 Fundamental data structures 19
3.2 Atom implementations 20
3.3 Problem class and Parameter updates 26
3.4 CVXPY interface 26

CHAPTER 4 Benchmark results 27
4.1 Benchmark Setup 27
4.1.1 Example 27
4.1.2 Runtime and Memory Usage 27
4.2 Results on Standard Benchmarks 27
4.3 Results on Stress Test Problems 27
4.3.1 Least Squares Problems 27

CHAPTER 5 CONCLUSION 28

5.1 Summary of Works 28
5.2 Limitations 28
5.3 Future Research 28

REFERENCES 29

LIST OF TABLES

LIST OF FIGURES

Figure 1.1 Example of expression tree for the function `log(|Ax-b|)`. 2

Figure 2.1 Users write functions in Python code that are restricted to be from the modeling language. Variables are labeled with a circle, and parameters with a square. 8

Figure 2.2 After the epigraph transformation reduction, the constraints and the objective are affine functions of the variables. 10

Figure 2.3 Matrix stuffing generates a 3D tensor for the constraint map. To depict these 3D tensors here, we leave the parameter values symbolic. Contrast this Figure with Fig. 1.1, which shows the operations being applied at each node. 11

Figure 2.4 Parameter map Q of the augmented objective $[c;\, d]$. The first n rows index the entries of the gradient c (one per post-canonicalization variable: t_1, t_2, and the n-block x); the final row indexes the constant offset d (zero here, since the canonicalized objective has no offset). Columns index the global augmented parameter vector $\tilde{\theta} = (g,\, \mu,\, 1)$. The g columns are zero because the objective does not depend on g, but they are part of $\tilde{\theta}$ globally and the analogous map for the constraints is non-zero in those columns. Multiplying Q by $\tilde{\theta}$ recovers $[c;\, d] = [1,\, \mu,\, 0,\, 0]^T$; the leading three entries match the c vector shown in Fig. 2.3. 12

Figure 2.5 Parameter map for $[A\, b]$ in the running example. The 3D tensor R has one 2D slice per entry of $\tilde{\theta} = (g_1, \ldots, g_m, \mu, 1)$. Block-row order matches Fig. 2.3(b): $x \in \mathbf{R}^n_+$; t_1 row of SOC_1; $Fx - g$ block of SOC_1; t_2 row of SOC_2; x block of SOC_2. Block-columns are the canonical-form variables $(t_1,\, t_2,\, x)$ followed by the b column. Summing $g_i \mathcal{R}[:, :, g_i]$ over i generates the $-g$ entry in b shown at the root of Fig. 2.3(b); the μ slice contributes nothing; the constant slice contributes everything else. 13

LIST OF SYMBOLS AND ACRONYMS

AD	Automatic Differentiation
AML	Algebraic Modeling Language
ASA	Affine-Solver-Affine
CSC	Compressed Sparse Column
CSR	Compressed Sparse Row
DAG	Directed Acyclic Graph
DCP	Disciplined Convex Programming
DPP	Disciplined Parameterized Programming
LP	Linear Program
MOI	MathOptInterface
MPS	Mathematical Programming System
NLP	Nonlinear Program
QP	Quadratic Program
SOCP	Second-Order Cone Program
SDP	Semidefinite Program

CHAPTER 1 INTRODUCTION

Domain specific modeling languages for convex optimization such as CVXPY [1] and YALMIP [2] allow users to formulate optimization problems in a natural mathematical syntax. Through a process called *canonicalization*, these high-level problem descriptions are converted to a *standard conic form* used by many general-purpose convex solvers

$$\begin{array}{ll} \text{minimize} & c^T x + d \\ \text{subject to} & b - Ax \in K \end{array} \tag{1.1}$$

where $x \in \mathbf{R}^n$ is the decision variables, and the *problem data* are the matrix $A \in \mathbf{R}^{m \times n}$, the vectors $b \in \mathbf{R}^m$ and $c \in \mathbf{R}^n$, the scalar $d \in \mathbf{R}$ and the non-empty closed convex cone $K \subseteq \mathbf{R}^m$. Almost all of optimization practice can be divided into two groups, *modeling* and *solving*. The modeler's role is to formulate problems based on real-world business or engineering applications. The solver's role is to find an optimal solution given the data of a problem instance. Modeling languages serve as the communication bridge between these two groups. When improvements in solvers are made, modeling languages must adapt to allow users to benefit from them. Conversely, it is through natural formulations that modelers can express their requirements and help motivate solver improvements.

While the time taken to solve an optimization problem is often dominated by the solver, the time taken to *canonicalize* (or *compile*) a problem can also be significant, especially for large problems. In the case of CVXPY, the *compilation time* largely lies in *matrix stuffing*, the final step of canonicalization where symbolic expression trees are converted to explicit numerical matrices. Other modeling languages take different approaches to form the problem data, we review these *coefficient extraction* approaches in §2.3.

Atoms. Modeling languages for convex optimization are built on top of a library of functions which can be used to describe problems. These functions are known as *atoms* in CVXPY since they are the fundamental building blocks of the modeling language [3]. Atoms have known attributes such as curvature, monotonicity, and sign, for example, .

Expression trees. An expression is recursively defined as an atom evaluated at a *subexpression*. The subexpression can either be a variable, a parameter, or another expression. Similarly to atoms, expressions can also be classified by curvature, monotonicity, and sign which are determined by simple *composition rules* [4]. An expression tree whose atoms are

all affine is known as an *affine expression tree*.

Parameterized conic form. The data inputs in a problem instance can either be fixed or varying. In CVXPY, varying data are referred to as *parameters* and fixed data are referred to as *constants*. Parameters are useful for specifying a *family* of problems where the structure is fixed [5]. A *parameterized cone program* extends the conic form in (**??**) by letting the problem data depend on a parameter vector $\theta \in \mathbf{R}^p$:

$$\begin{array}{ll} \text{minimize} & c(\theta)^T x + d(\theta) \\ \text{subject to} & b(\theta) - A(\theta)x \in \mathcal{K}. \end{array} \tag{1.2}$$

When c, d, b and A are *affine with respect to* θ, we can represent the problem data parameter dependence as

$$\begin{bmatrix} c \\ d \end{bmatrix} = Q \cdot \tilde{\theta}, \qquad \text{and} \qquad [A \quad b] = \sum_{i=1}^{p+1} \mathcal{R}[:,:,i] \cdot \tilde{\theta}_i, \tag{1.3}$$

where $Q \in \mathbf{R}^{(n+1)\times(p+1)}$, $\mathcal{R} \in \mathbf{R}^{m\times(n+1)\times(p+1)}$ and $\tilde{\theta} = (\theta, 1) \in \mathbf{R}^{p+1}$. When $A(\theta)$ is sparse, we also require that its structure or *sparsity pattern* must be fixed throughout the different iterations. We refer to Q as the objective *parameter map*, $\mathcal{R}$ as the constraint parameter map, and $\tilde{\theta}$ as the extended parameter vector. The equation for the constraint map is known as a *tensor* product, a generalization of matrix-vector product in higher dimensions. This notation is required to output a constraint matrix from a three dimensional tensor.

Matrix stuffing. Although the original user specified problem formulation may contain nonlinearities, after conversion to standard form, the *canonicalized problem* will only contain affine expressions (with respect to both parameters and variables). Concretely, matrix stuffing refers to the task of forming the constraint parameter map, as the objective parameter map is a scalar special case of it. Once the parameter maps are formed, the parameter depen-

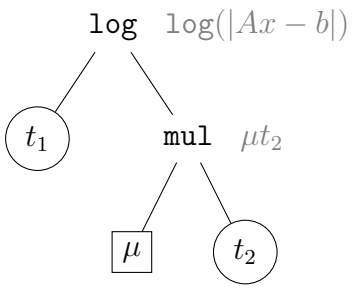


Figure 1.1 Example of expression tree for the function `log(|Ax-b|)`.

dence of the model is *cached* and doesn't rely on the expression tree representation anymore. Subsequent problem instances will only need to update θ and apply the maps to get the new problem data. We review current CVXPY matrix stuffing approaches and give examples of how the parameter maps are formed in §2.2.

Contributions. We make the following contributions:

1. We propose an alternative method to matrix stuffing that doesn't rely on forming parameter maps. Instead, we treat matrix stuffing as a sparse Jacobian evaluation which naturally separates the symbolic structure of the Jacobian from its numerical values.

2. We use the affine atoms from the `SparseDiffengine` package and its Python bindings to implement a canonicalization backend for CVXPY.

3. We benchmark our method against existing approaches and make improvements to CVXPY's codebase.

All code is open source and available at `https://github.com/cvxpy/cvxpy`.

Thesis outline. The rest of this thesis begins with a review of the literature on modeling languages, canonicalization, and matrix stuffing in §2. In §3 we describe a set of relevant affine atoms and their implementation in a sparse differentiation package. In §4 we present benchmark results comparing our method to existing approaches. Finally, in §5 we summarize our contributions and discuss future improvements to the canonicalization backend.

CHAPTER 2 LITERATURE REVIEW

Solving an optimization problem on a computer requires translating the mathematical formulation into a data format that solvers can process. For linear programs, this means constructing the constraint matrix A, the right-hand side vector b, and the objective coefficients c that define the *standard form*,

$$\begin{aligned} \text{minimize} \quad & c^T x \\ \text{subject to} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned} \tag{2.1}$$

Manual encoding In the early days of linear programming, analysts manually computed and encoded each coefficient onto punch cards. This process was tedious, error-prone and became impractical as linear programming found more and more applications in large-scale industrial problems [6].

MPS format IBM's Mathematical Programming System (MPS) file format, developed in the 1960s, established a standardized file format for representing linear programs [7]. The MPS format uses fixed-width fields designed for 80-column punch cards, with sections for rows, columns, and coefficients. William Orchard-Hays [8] provides a comprehensive history of these early mathematical programming systems, tracing their development from 1952 at RAND Corporation through their widespread adoption on mainframe computers in the late 1960s.

Matrix generators To automate the creation of MPS files, *matrix generators* emerged in the late 1960s; they are procedural programs that compute the problem coefficients through explicit loops and output the results in MPS format [9]. Commercial systems such as `MaGen` [10], `GAMMA` [11], and `DATAFORM` [12] were among the first matrix generators which were used in various domains.

A modern approach Modelers naturally think in terms of sets, indices, and algebraic expressions, not in terms of individual matrix entries. Matrix generators simplified the encoding process but still required modelers to translate their mathematical formulations into procedural code that explicitly constructs the coefficient matrices. This suggests that the gap between the modeler's representation of the problem and the solver's required format

hadn't been resolved yet [6].

Algebraic modeling languages In the late 1970s, *algebraic modeling languages* emerged as formal languages designed to accept problem descriptions that closely mirror standard mathematical notation [13]. By working directly with the modeler's representation, these languages simplify verification, modification, and communication. In the late 1980s and early 1990s, commercial algebraic modeling systems such as GAMS [14], AIMMS [15] and AMPL [16] became widely adopted in both academia and industry (and they are still widely used today).

Embedded modeling languages In the late 2000s and early 2010s, modeling languages emerged that are embedded in general-purpose programming languages (in contrast with standalone systems like AMPL and GAMS). Examples include YALMIP [2] in MATLAB, Pyomo [**?**] in Python, JuMP [17] in Julia, and CasADi [18] in C++.

Disciplined convex programming In addition, there are domain specific languages that implement *disciplined convex programming* (DCP), a set of composition rules that can automatically verify convexity [3, 4]. These include CVX [19] in MATLAB, CVXPY [1, 20] in Python, and Convex.jl [21] in Julia.

Structure of the chapter The sections on canonicalization and matrix stuffing will be about CVXPY, the Python implementation of DCP. The reasons are twofold: it is the most updated implementation which still has an active community and the work of this thesis aims to be integrated in CVXPY. In section **??**, we present the embedded modeling language JuMP and one of the original algebraic languages AMPL. In doing so, we also compare the differences with approaches taken in CVXPY. Both modeling languages treat the conic problem data as a form a sparse Jacobian computation so we review approaches on computing derivatives in **??**. Finally, all modeling languages use sparse matrix formats to store the problem data matrices, they are incredibly useful and we review them in **??**.

2.1 Canonicalization

CVXPY. CVXPY is a domain-specific language for convex optimization embedded in Python, that was originally developed by Steven Diamond and Stephen Boyd at Stanford University [1]. Akshay Agrawal later redesigned the rewriting system in CVXPY [20], which

enabled support for other disciplined languages [22–24], and the project has since grown through contributions from many others [25].

Functional form. When a user specifies a CVXPY problem, it is usually in a "functional" form

$$\begin{array}{ll} \text{minimize} & f_0(x;\theta) \\ \text{subject to} & f_i(x;\theta) \leq g_i(x;\theta),\ i = 1,\ldots,m_1, \\ & h_i(x;\theta) = j_i(x;\theta),\ i = 1,\ldots,m_2, \end{array} \tag{2.2}$$

with variable $x \in \mathbf{R}^n$ and parameter vector $\theta \in \mathbf{R}^p$ [26]. For a given value of θ we refer to (2.2) as a problem instance, otherwise for general values of θ it defines a problem family. Each function in (**??**) is internally represented as an expression tree.

Disciplined convex programming. When all f_i are convex, all g_i are concave and all h_i and j_i are both affine, the functional form defines a convex optimization problem. To verify that the problem is convex, CVXPY uses the DCP composition rules to classify the curvature of functions. Once all functions are classified, a problem is *DCP-compliant* if its functions satisfies the curvatures of a convex problem. Otherwise, if certain curvatures are incorrect (*i.e.* f_0 is concave) or unknown, CVXPY will raise an error and terminate.

Composition rules. The expression $h(f_1(x), f_2(x), \ldots, f_k(x))$ is convex when h is convex and for each $i \in \{1,\ldots,k\}$ the following holds:

- h is non-decreasing in argument i and f_i is convex, or
- h is non-increasing in argument i and f_i is concave, or
- f_i is affine.

The same results hold for concave functions by swapping convex and concave. Affine expressions require h and all of its arguments to be affine [3].

DPP composition rules. *Disciplined parameterized programming* (DPP) is an extension of DCP which guarantees an affine parameter dependence in the problem data. A problem is *DPP-compliant* if it satisfies the DPP composition rules. The rules essentially disallows products of expressions involving parameters and can be referred to in [24].

Solver standard form. After a problem is verified to be DPP-compliant, CVXPY will convert the functional form problem to a target solver's standard form. CVXPY supports solvers for quadratic programming (such as OSQP and HiGHS), conic programming (such as ECOS), and conic quadratic programming (such as Mosek and Clarabel). The conic quadratic form is the most general of these, but is not a major generalization of the cone form: any conic quadratic program can be reduced to an equivalent cone program by linearizing the quadratic objective via the second-order cone, at the cost of one extra variable and one extra constraint. We will focus on the conic form throughout the thesis.

Reductions. Canonicalization converts a DPP-compliant model into a cone program via a sequence of reductions [20].

Maximization problems. A first example of a reduction is one which takes a maximization problem and converts it into a minimization problem by flipping the sign of the objective. The reverse transformation simply undoes the sign flipping and has no effect on the actual solution of the problem. In CVXPY, this reduction is conveniently called `FlipObjective`.

Variable bounds. A second example is variable attributes such as numerical bounds or `nonneg`. During canonicalization, CVXPY will also verify if the target solver supports bounds natively. If it does, CVXPY can pass them directly to solvers, otherwise they will get converted into explicit constraints. The reverse transformation for this reduction is to take the dual values corresponding to the explicit constraints ($x \geq 0$) and setting them to be equal to the variable bound's dual values. In CVXPY, this reduction is part of a greater set of general attribute constraints and is called `CvxAttr2Constr`.

Epigraph transformation. Each convex atom has a *canonicalization* that replaces it with an epigraph variable (for concave atoms it is a *hypograph* variable) and auxiliary constraints of the form $f(x, t, \theta) \in \mathcal{K}$ where t is the epigraph variable, f is affine in the variables (x, t) for fixed parameters θ, and $\mathcal{K}$ is a convex cone. This process was presented initially in [**?**] One illustrative canonicalization replaces the `norm2` expressions in the objective with a new epigraph variable t_i and imposes the epigraph constraint what is known as a *second-order* cone

$$(t_i, z_i) \in \mathcal{K}_{\text{soc}}. \tag{2.3}$$

Nondifferentiability.

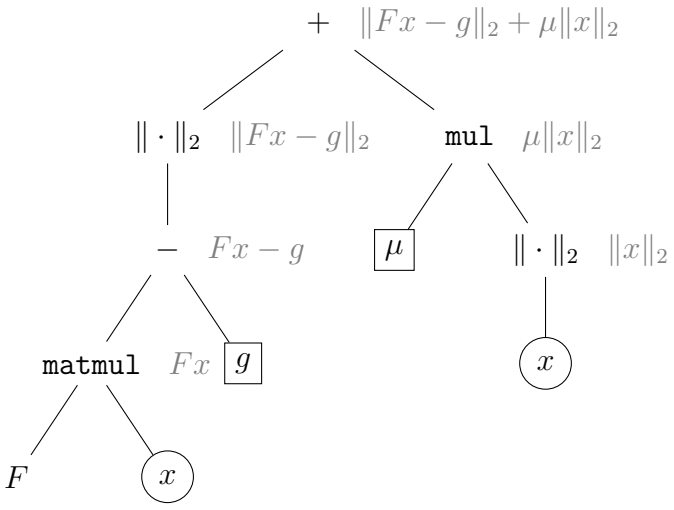
Example of canonicalization. To show a concrete example of canonicalization, we consider a simple fitting with regularization problem

$$\begin{array}{ll} \text{minimize} & \|Fx - g\|_2 + \mu\|x\|_2 \\ \text{subject to} & x \succeq 0 \end{array} \tag{2.4}$$

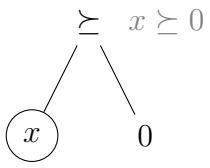
with variable $x \in \mathbf{R}^n$, fixed problem data $F \in \mathbf{R}^{m\times n}$, and parameters $g \in \mathbf{R}^m$ and $\mu \geq 0$. The problem formulation in CVXPY code is

```
import cvxpy as cp
import numpy as np
F = np.random.randn(m, n)
x = cp.Variable(n)
g = cp.Parameter(m)
mu = cp.Parameter(nonneg=True)
objective = cp.Minimize(cp.norm(F @ x - g) + mu * cp.norm(x))
prob = cp.Problem(objective, [x >= 0])
```

where we also use the numerical computation library `NumPy` [**?**] for the data arrays. The expression tree representation of the problem is shown in **??**.



(a) Abstract syntax tree of objective



(b) Abstract syntax tree of constraints

Figure 2.1 Users write functions in Python code that are restricted to be from the modeling language. Variables are labeled with a circle, and parameters with a square.

Conic form expression tree. After applying the reductions, the problem has been *canonicalized* to

$$\begin{array}{ll} \text{minimize} & t_1 + \mu t_2 \\ \text{subject to} & x \in \mathbf{R}^n_+ \\ & (t_1, Fx - g) \in \mathcal{K}_{\text{soc},m+1} \\ & (t_2, x) \in \mathcal{K}_{\text{soc},n+1} \end{array} \tag{2.5}$$

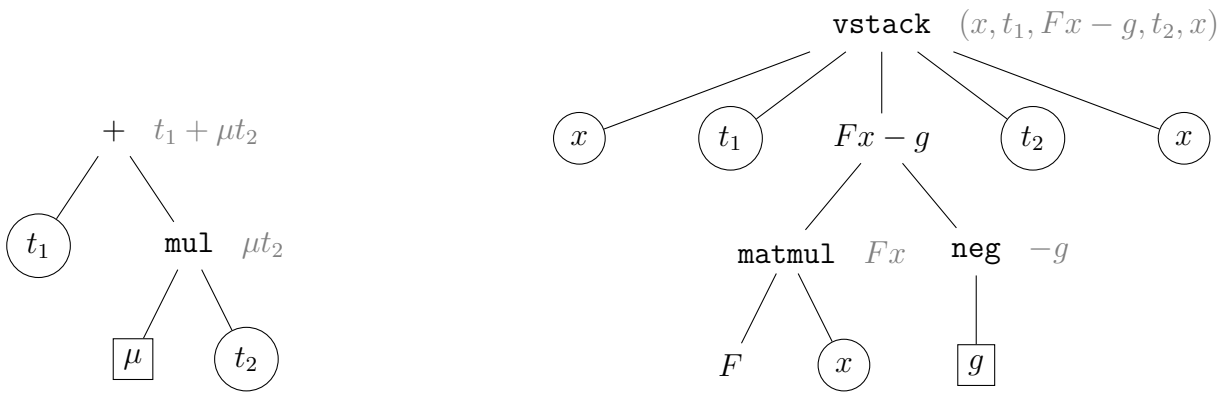
where t_1 and t_2 are new epigraph variables, and the cone $\mathcal{K}$ from **??** is the product $\mathbf{R}^n_+ \times \in \mathcal{K}_{\text{soc},m+1} \times \mathcal{K}_{\text{soc},n+1}$. In the next section, we explain how the symbolic affine expression trees in **??** get converted into the parameter maps defined in **??**.

References. Michael Grant's papers and thesis on CVX and DCP are the standard references on verifying convexity and canonicalization [**?**, 3, 4]. The paper by Chu et al. [27] gives another perspective of canonicalization through the *relaxed smith form*, it was used extensively to learn the concept initially. The paper by Agrawal et al. [20] on rewriting systems is a great reference for understanding reductions and the core concepts behind CVXPY. Finally, the `CVXPY` documentation and the Mosek modeling cookbook can be used as complementary references for epigraph reformulations and general modeling principles.

2.2 Matrix stuffing

In this section, we explain how to form the parameter maps from a set of affine expression trees. In doing so, we also cover fundamental data structures and design choices of these backends which can later be contrasted with the method proposed in **??**. Historically, the implementations of matrix stuffing have been referred to as *canonicalization backends*, and there has been many versions over the years. We will focus on `cvxcore`, one of the original implementations in `C++`, and `SciPy`, an improved Python version which relies on the sparse matrix library with the same name.

Tensor object. Mathematically, tensors are a generalization of matrices in three dimensions or higher. In the canonicalization backends, this abstraction is also relevant but can be made more precise. We consider a `Tensor` as a three-dimensional *sparse* array which represents a sparse linear mapping from a parameter vector to the constraint matrix (as shown in **??**). The three indices are rows for constraints, columns for variables, and parameter offsets for parameters.



(a) Expression tree of the objective

(b) Expression tree of the constraints

Figure 2.2 After the epigraph transformation reduction, the constraints and the objective are affine functions of the variables.

Recursive walk. Fundamentally, matrix stuffing is relatively simple; it requires a single depth-first walk of the affine expression trees, where every node applies an linear operation on a `Tensor`. Since the process is recursive, every node of an expression tree will have an equivalent `Tensor`, and the final output for $\mathcal{R}$ is the `Tensor` at the root node.

Local vs. Global problem data. All modeling languages for optimization must atleast do one thing correctly: abstract away the task of placing *local* tensors (or its equivalent) at the expression level (where typically a single variable appears) towards a *global* tensor at the problem level (where many more variables could be present). In fact, once we are able to compute the local tensor for all expressions of interest, getting the global tensor simply comes down to combining the local tensors and applying appropriate row and column *offsets*.

Column offsets.

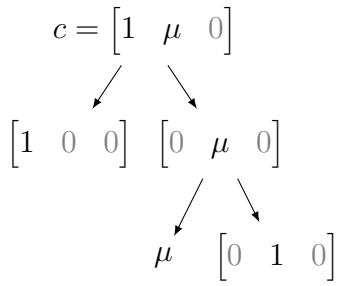
`InverseData` for example.

Data structures.

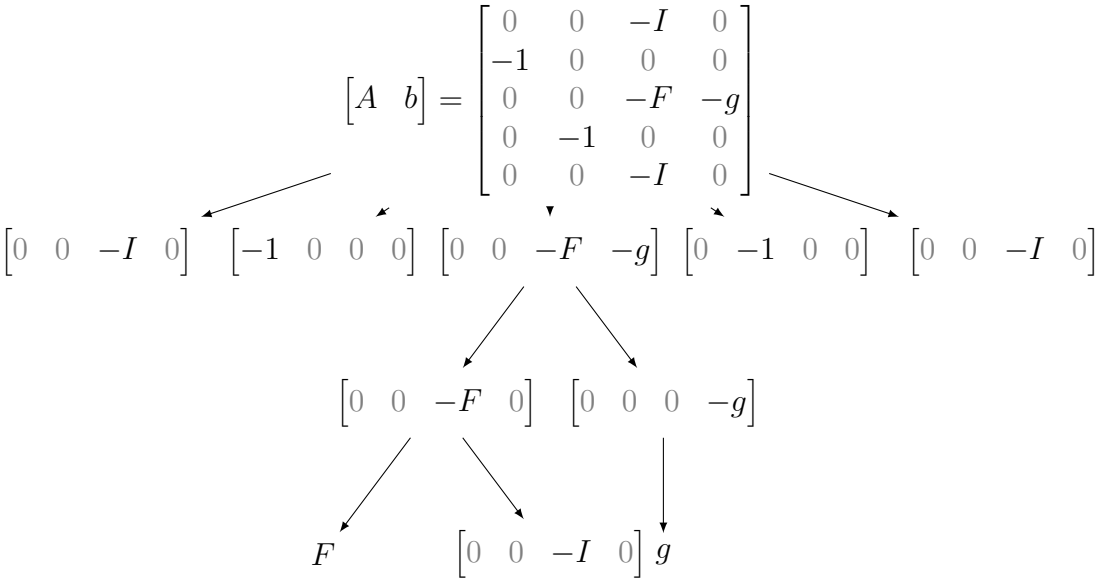
Limitations.

Linear operator.

Tensor Representation.



(a) Matrix stuffing of the objective



(b) Matrix Stuffing tree

Figure 2.3 Matrix stuffing generates a 3D tensor for the constraint map. To depict these 3D tensors here, we leave the parameter values symbolic. Contrast this Figure with Fig. 1.1, which shows the operations being applied at each node.

$$Q = \begin{array}{c} \\ t_1 \\ t_2 \\ x \\ d \end{array} \begin{array}{c} \begin{array}{ccc} g^T & \mu & 1 \end{array} \\ \begin{bmatrix} 0 \cdots 0 & 0 & 1 \\ 0 \cdots 0 & 1 & 0 \\ 0 \cdots 0 & 0 & 0 \\ 0 \cdots 0 & 0 & 0 \end{bmatrix} \end{array} \qquad \begin{bmatrix} c \\ d \end{bmatrix} = Q\,\tilde{\theta} = \begin{bmatrix} 1 \\ \mu \\ 0 \\ 0 \end{bmatrix}$$

Figure 2.4 Parameter map Q of the augmented objective $[c;\, d]$. The first n rows index the entries of the gradient c (one per post-canonicalization variable: t_1, t_2, and the n-block x); the final row indexes the constant offset d (zero here, since the canonicalized objective has no offset). Columns index the global augmented parameter vector $\tilde{\theta} = (g,\, \mu,\, 1)$. The g columns are zero because the objective does not depend on g, but they are part of $\tilde{\theta}$ globally and the analogous map for the constraints is non-zero in those columns. Multiplying Q by $\tilde{\theta}$ recovers $[c;\, d] = [1,\, \mu,\, 0,\, 0]^T$; the leading three entries match the c vector shown in Fig. 2.3.

References.

2.3 Coefficient extraction

`JuMP.` JuMP is an algebraic modeling language embedded in Julia, originally developed in 2012 by Iain Dunning, Joey Huchette, and Miles Lubin at the MIT Operations Research Center [17, 28]. It is now being developed and maintained full-time by Oscar Dowson.

`MathOptInterface`

`CasADi.` [18]

`AMPL.` [16]

References.

2.4 Computing derivatives

Finding the rate of change or the *derivative* of a function at a particular point is a fundamental concept in calculus that was discovered by Leibniz and Newton in the late 1600s. Since then, although the underlying theory has practically remained the same, various methods have been developed to efficiently compute derivatives in different contexts and applications. More recently, the rise of machine learning and deep learning brought renewed interest in this age-old problem through a new application of derivatives in training *neural networks*,

$$\mathcal{R}[:,:,g_i] = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -e_i \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \qquad \mathcal{R}[:,:,\mu] = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

(a) Parameter slices. Each $\mathcal{R}[:,:,g_i]$ $(i = 1, \ldots, m)$ places a single -1 in the b column at the i-th row of the $Fx - g$ block (denoted $-e_i$). $\mathcal{R}[:,:,\mu] = 0$ because μ does not appear in any constraint.

$$\mathcal{R}[:,:,1] = \begin{bmatrix} 0 & 0 & -I & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & -F & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -I & 0 \end{bmatrix}$$

(b) Constant slice. Holds the fixed structure: $-I$ blocks for the variable bound and SOC_2 rows, -1's for the epigraph variables, and $-F$ for the Fx block.

Figure 2.5 Parameter map for $[A\, b]$ in the running example. The 3D tensor R has one 2D slice per entry of $\tilde{\theta} = (g_1, \ldots, g_m, \mu, 1)$. Block-row order matches Fig. 2.3(b): $x \in \mathbf{R}^n_+$; t_1 row of SOC_1; $Fx - g$ block of SOC_1; t_2 row of SOC_2; x block of SOC_2. Block-columns are the canonical-form variables (t_1, t_2, x) followed by the b column. Summing $g_i\mathcal{R}[:,:,g_i]$ over i generates the $-g$ entry in b shown at the root of Fig. 2.3(b); the μ slice contributes nothing; the constant slice contributes everything else.

it is known as the *backward pass* or *backpropagation* [29, 30]. Some theorists will claim that backpropagation is little more than a clever application of the *chain rule* [31], but the large scale of modern workloads has led to the development of incredible software tools and implementation techniques which are interesting in their own right. This section provides an overview of these new tools and techniques, with a particular emphasis on *algorithmic differentiation* (AD) and its applications to computing sparse Jacobians [32].

Numerical differentiation. Other than computing derivatives by hand, the simplest method is to use *numerical differentiation*, which approximates the derivative of a function at a point by evaluating the function at nearby points using a *finite difference* formula similar to either

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}, \tag{2.6}$$

or

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}, \tag{2.7}$$

where h is a small step size. Although this approach is straightforward to implement and can achieve reasonable accuracy, there are important downsides to consider. First, it requires a linear number of function evaluations with respect to the number of input variables, which is expensive for high-dimensional functions. Second, it can easily lead to *truncation errors* (when h is too large) or *rounding errors* (when h is too small) in the computed derivative if the step size is not carefully chosen [33]. Despite these downsides, numerical differentiation is still widely used in practice, especially for verifying the correctness of derivative computations (also known as a *derivative checker*) [34].

Symbolic differentiation. Mathematical expressions can be manipulated automatically using simple derivative rules of calculus, such as the product and sum rule. This *symbolic* approach to differentiation is used in computer algebra systems such as Mathematica [35] and Maple [36] as well as the SymPy library in Python [37]. Unlike its numerical counterpart, symbolic differentiation tools compute derivatives as closed-form expressions, which can be simplified and evaluated at any arbitrary point. These closed-form expressions are particularly useful for deriving analytic solutions of optimization problems, and can be further used in mathematical proofs and analyses.

However, a naive implementation of symbolic differentiation can easily lead to exponentially large symbolic expressions, which is impractical and inefficient for many problems [32]. Interestingly, when intermediate expressions are stored as *common subexpressions*, and the expression is represented with a *directed acyclic graph* (DAG) instead of a tree, symbolic differentiation performs the same set of operations as *reverse mode differentiation* [38].

Algorithmic differentiation. There is a lot of confusion on the terminology surrounding the various methods for computing derivatives; *automatic differentiation* and *computational differentiation* are often used to describe the field, but we will instead refer to it as *algorithmic differentiation* since it emphasizes the algorithms behind the different implementations rather than a particular method (it is also the convention in the seminal book by Griewank and Walther [32]). Fundamentally, AD is a non-standard interpretation of a computer program: it augments the *evaluation trace* of a function (the sequence of floating point values calculated during the execution) with calculations of its derivatives [39]. This trace can be represented as a *computational graph*, a DAG in which each node corresponds to an elementary operation with known derivatives. Combining the derivatives of these operations throughout the graph using the chain rule gives the derivative of the overall function [32]. The computational graph is the central data structure of AD; the various *modes* of AD correspond to different ways elementary derivatives are accumulated on this graph.

Forward mode AD. The method of propagating tangents forward through the computational graph is known as *forward mode* AD. A single forward pass simultaneously computes the value and derivative of every intermediate variable with respect to a single input variable, giving the sensitivity of all outputs to that input. The cost of a forward pass is a small constant factor of the cost of evaluating the original function, since each elementary operation is augmented with at most a few additional arithmetic operations for the derivatives. However, one downside of forward mode AD is that the number of forward passes scales linearly with the number of input variables, which is impractical for certain functions (which have many input variables and very few output variables).

Reverse mode AD. In contrast to forward mode, *reverse mode* AD propagates adjoints backward through the computational graph. It first requires a forward pass to compute and store the value of every intermediate variable, and it then performs a reverse pass to compute the sensitivities of a single output variable with respect to all input variables. The cost of a reverse pass is also a small constant factor of the cost of evaluating the original function, but the number of passes required now scales linearly with the number of output variables rather than the number of input variables. This makes reverse mode AD particularly efficient for computing gradients of scalar-valued functions, making it the default method for modern machine learning libraries such as Jax [40], Pytorch [41], and Tensorflow [42].

Advantages of AD. [43].

Jacobian and Hessian computation.

Sparse Jacobian computation. graph coloring, cross-country elimination, vertex elimination, jacobian accumulation, etc. maybe mention graph coloring here?

[44] presents a code generator for operations on sparse data.

[45] presents a similar method for computing sparse Hessians with graph coloring. [46] is a survey of graph coloring methods for computing sparse Jacobians and Hessians.

Differentiation packages. mention ADOL-C and CppAD for C++. mention AMPL and CasADi for optimization. and also mention that they were covered above in the coeff extraction section.

References. The book by Griewank and Walther [32] is a comprehensive reference on the theory and practice of automatic differentiation. The book by Naumann [43] is another great reference that focuses on the implementation of AD in software, with a particular emphasis on the use of operator overloading and source code transformation techniques. [39] is an excellent and relatively up-to-date survey of the various methods for automatic differentiation in machine learning.

2.5 Sparse matrices

Matrices that contain many number of zero entries are known as *sparse matrices*. The simplest sparse matrix data structure is the *coordinate* or *triplet* format. It consists of two integer index arrays `rows` and `cols` and a floating point value array `vals` of length equal to the number of *nonzero entries* in the matrix.

For example, the matrix

$$\begin{bmatrix} 0 & 0 & 3.2 & 0 \\ 4.1 & 1.8 & 0 & 2.2 \\ 0 & 2.8 & 0 & 3.4 \end{bmatrix} \tag{2.8}$$

can be represented in coordinate format as

```
int    rows[] = {0,  1,   1,   1,   2,   2};
int    cols[] = {2,  0,   1,   3,   1,   3};
double vals[] = {3.2, 4.1, 1.8, 2.2, 2.8, 3.4};
```

where we use zero-based indexing and array declaration notation from the C programming language [47]. The coordinate format is the easiest to understand and form, but it is not effective for most operations on sparse matrices; the *compressed sparse row* or CSR format is commonly used instead.

Compressed sparse row format. The CSR format differs from the coordinate format in that it replaces the `rows` array with a row pointer array `idxptr` of length equal to the number of rows plus one. The row pointer array indicates the starting index of each row in the `cols` arrays. For example, the matrix in (2.8) can be represented in CSR format as

```
int  idxptr[] = {0,  1,              4,        6};
int   cols[] = {2,  0,   1,   3,   1,   3};
double vals[] = {3.2, 4.1, 1.8, 2.2, 2.8, 3.4};
```

where we deliberately format `idxptr` to align with the start of each row in the `cols` arrays. The benefits of the CSR format are more than a reduced size of the `idxptr` array compared

to the `rows` array. In the CSR format, accessing a row takes constant time and is very *cache-friendly* since the nonzero entries are stored contiguously in memory. For this reason, the CSR format is the default for *iterative methods* to solve sparse linear systems [48].

Compressed sparse column format. However, one downside of the CSR format is that accessing a column is much more difficult; one would need to look at every row to find the nonzero entries in a column. An alternative format is the *compressed sparse column* or CSC format, which swaps the role of the `idxptr` array to refer to column indices instead. In contrast to iterative methods, the CSC format is the preferred format for *direct methods* to solve sparse linear systems [49].

Transpose.

Which format to use?

Other formats. The coordinate, CSR and CSC formats are the most widely used sparse matrix formats, but many others exist for different specialized use cases. The scientific Python sparse library `scipy.sparse` [50] allows users to form other types of sparse matrices such as the list of lists (LIL) matrix, the dictionary of keys (DOK) matrix, and the block-sparse row (BSR) matrix.

Sparse matrix struct in C. A CSR matrix struct in C looks like this.

```
typedef struct CSR_Matrix {
    int *p;      /* row pointers, size m+1 */
    int *i;      /* column indices, size nnz */
    double *x;   /* values, size nnz       */
    int m, n, nnz;
} CSR_Matrix;
```

References. Although the work in this thesis does not necessarily revolve around solving sparse linear systems, a lot of techniques for manipulating sparse matrices we use were inspired by the literature. In particular, the book by Davis [49] was used extensively throughout the development of our methods. The book by Duff [51] is another great reference to learn more about the history and developement of sparse matrix algorithms. Another reference is the review article by the "gang of four" (Gill, Murray, Saunders and Wright) from the

Systems Optimization Laboratory at Stanford University [52] which provides a high-level overview of various applications of sparse matrix methods in optimization.

CHAPTER 3 A lightweight canonicalization engine

In this chapter we present a minimal implementation of a subset of affine atoms in the `SparseDiffEngine` package, and show how they can be used to compute the sparse constant Jacobian of a set of affine expressions. At this point, we are given a *canonicalized* problem in standard form, after the canonicalization process described in §2.1. The content of this chapter has many similarities to §2.2, since both serve the same purpose of forming the problem data. The structure of the chapter is as follows: in §3.1 we describe basic data structures in the engine which will be used throughout the atom implementations described in §3.2, and finally, in §3.4 we present a high level problem class interface which can easily be used in CVXPY. We also recommend reading §2.5 before reading this chapter, since it covers the basics of sparse matrices and their data structures.

3.1 Fundamental data structures

The main data structure in the engine is the expression struct.

```
typedef struct expr {
    int   d1, d2, size; /* shape (d1 x d2) and size = d1*d2 */
    int   n_vars;       /* total number of variables     */
    int   var_id;       /* offset into u, or NOT_A_VARIABLE */

    struct expr *left;  /* left child (NULL for leaves)  */
    struct expr *right; /* right child (NULL for leaves) */

    double    *value;   /* forward-pass output, size entries */
    CSR_Matrix *jacobian; /* size x n_vars sparse Jacobian */

    forward_fn     forward;
    jacobian_init_fn jacobian_init_impl;
    eval_jacobian_fn eval_jacobian;
} expr;
```

every expression node has a shape (d1 x d2) and a size (d1*d2), and also two pointers to its left and right children. It will also have a pointer to its value, which is the output of the forward pass, and a pointer to its Jacobian, which is a CSR matrix. Every expression must also implement three functions: a forward function, a Jacobian initialization function, and a

Jacobian evaluation function.

Leaf nodes. The two leaf nodes are variables and parameters. They are the fundamental building blocks of the expression tree.

3.2 Atom implementations

We will now show the implementation of atoms in the engine. The semantics of an atom is as follows, it takes one or two expressions as input, and produces an output expression corresponding to the result of the operation. Every atom implementation below will first show the basic CVXPY code, and then show how it is implemented in the engine.

Helpers and definitions. We first import the necessary libraries and define basic variables and constants which will be used throughout the implementation.

```
import cvxpy as cp
import numpy as np
import scipy.sparse as sp

x = cp.Variable(n1)
y = cp.Variable(n2)
a = cp.Parameter(n1)
b = cp.Parameter(n2)
```

Addition. The addition atom (also commonly referred to as `add`) takes two expressions and sums them together elementwise. The CVXPY code is as follows (note that Python's operator overloading for `+` will automatically form an `add` atom).

```
add_expr = cp.add(x, y)
```

In the engine the implementation is as follows.

```
static void forward(expr *node, const double *u)
{   /* forward pass for children */
    node->left->forward(node->left, u);
    node->right->forward(node->right, u);
    for (int i = 0; i < node->size; i++)
    {
        node->value[i] = node->left->value[i] + node->right->value[i];
```

```
    }
}

static void jacobian_init_impl(expr *node)
{
    /* initialize children's jacobians */
    jacobian_init(node->left);
    jacobian_init(node->right);

    /* we never have to store more than the sum of children's nnz */
    int nnz_max = node->left->jacobian->nnz + node->right->jacobian->nnz;
    node->jacobian = new_csr_matrix(node->size, node->n_vars, nnz_max);

    /* fill sparsity pattern */
    sum_csr_alloc();
}

static void eval_jacobian(expr *node)
{
    /* evaluate children's jacobians */
    node->left->eval_jacobian(node->left);
    node->right->eval_jacobian(node->right);

    /* sum children's jacobians */
    sum_csr_fill_values();
}
```

Negation. The negation atom takes a single expression and negates it elementwise.

Transpose. The transpose atom takes a single expression and transposes it. In CVXPY the code is as follows (it can also be written as `x.T`).

```
transpose_expr = cp.transpose(x)
```

```
// Forward pass for transpose atom
static void forward(expr *node, const double *u)
{
    /* forward pass for child */
    node->left->forward(node->left, u);
```

```
    /* local forward pass */
    int d1 = node->d1;
    int d2 = node->d2;
    double *X = node->left->value;
    double *XT = node->value;
    for (int i = 0; i < d1; ++i)
    {
        for (int j = 0; j < d2; ++j)
        {
            XT[j * d1 + i] = X[i * d2 + j];
        }
    }
}
```

explain the Jacobian init and eval for transpose, which is just a permutation of the Jacobian entries.

Horizontal stack. The horizontal stack atom takes two expressions and concatenates them horizontally. In CVXPY the code is as follows, and just like `NumPy`, it requires the input expressions to have the same number of rows.

```
hstack_expr = cp.hstack([x, y])
```

```
static void forward(expr *node, const double *u)
{
    hstack_expr *hnode = (hstack_expr *) node;

    /* children's forward passes */
    for (int i = 0; i < hnode->n_args; i++)
    {
        hnode->args[i]->forward(hnode->args[i], u);
    }

    /* concatenate values horizontally */
    int offset = 0;
    for (int i = 0; i < hnode->n_args; i++)
    {
        expr *child = hnode->args[i];
```

```
        memcpy(node->value + offset, child->value, child->size * sizeof(double
            ));
        offset += child->size;
    }
}

static void eval_jacobian(expr *node)
{
    hstack_expr *hnode = (hstack_expr *) node;
    CSR_Matrix *A = node->jacobian;
    A->nnz = 0;

    for (int i = 0; i < hnode->n_args; i++)
    {
        expr *child = hnode->args[i];
        child->eval_jacobian(child);

        /* copy values */
        memcpy(A->x + A->nnz, child->jacobian->x,
               child->jacobian->nnz * sizeof(double));
        A->nnz += child->jacobian->nnz;
    }
}
```

Vertical stack. The vertical stack atom takes two expressions and concatenates them vertically. In CVXPY the code is as follows, and just like `NumPy`, it requires the input expressions to have the same number of columns.

```
vstack_expr = cp.vstack([x, y])
```

In the engine the implementation uses the transpose and horizontal stack atoms, since we can write `vstack([x, y]) = transpose(hstack(transpose(x), transpose(y)))`.

More generally, we have the following equivalence:

```
vstack([A, B]) == transpose(hstack([transpose(A), transpose(B)]))
```

```
index_expr = x[0:3]
```

```
static void forward(expr *node, const double *u)
```

```
{
    expr *x = node->left;
    index_expr *idx = (index_expr *) node;

    /* child's forward pass */
    x->forward(x, u);

    /* gather selected elements */
    for (int i = 0; i < idx->n_idxs; i++)
    {
        node->value[i] = x->value[idx->indices[i]];
    }
}
```

mention that the Jacobian is a subset of the child's Jacobian, and we can fill it in by copying the relevant rows.

```
sum_expr = cp.sum(x)

static void forward(expr *node, const double *u)
{
    /* child's forward pass */
    x->forward(x, u);
    sum = 0.0;
    end = x->d1 * x->d2;
    /* sum all elements */
    for (i = 0; i < end; i++)
    {
        sum += x->value[i];
    }
    node->value[0] = sum;
}
```

```
elemwise_mult_expr = cp.multiply(x, y)

static void forward(expr *node, const double *u)
```

```
{
    expr *child = node->left;
    vector_mult_expr *vnode = (vector_mult_expr *) node;

    /* call forward for param_source expr tree (this extra logic is needed
       in case the parameter is a broadcast or promote node which needs to
          refresh
       its values) */
    if (vnode->base.needs_parameter_refresh)
    {
        vnode->param_source->forward(vnode->param_source, NULL);
        vnode->base.needs_parameter_refresh = false;
    }

    const double *a = vnode->param_source->value;

    /* child's forward pass */
    child->forward(child, u);

    /* local forward pass: elementwise multiply by a */
    for (int i = 0; i < node->size; i++)
    {
        node->value[i] = a[i] * child->value[i];
    }
}

static void jacobian_init_impl(expr *node)
{
    expr *x = node->left;

    /* initialize child jacobian */
    jacobian_init(x);

    /* same sparsity as child */
    node->jacobian = new_csr_copy_sparsity(x->jacobian);
}

static void eval_jacobian(expr *node)
```

```
{
    expr *x = node->left;
    const double *a = ((vector_mult_expr *) node)->param_source->value;

    /* evaluate x */
    x->eval_jacobian(x);

    /* row-wise scale child's jacobian */
    for (int i = 0; i < node->size; i++)
    {
        for (int j = x->jacobian->p[i]; j < x->jacobian->p[i + 1]; j++)
        {
            node->jacobian->x[j] = a[i] * x->jacobian->x[j];
        }
    }
}
```

Matrix multiplication.

3.3 Problem class and Parameter updates

3.4 CVXPY interface

CHAPTER 4 Benchmark results

4.1 Benchmark Setup

4.1.1 Example

4.1.2 Runtime and Memory Usage

4.2 Results on Standard Benchmarks

4.3 Results on Stress Test Problems

4.3.1 Least Squares Problems

CHAPTER 5 CONCLUSION

Texte / Text.

5.1 Summary of Works

5.2 Limitations

5.3 Future Research

REFERENCES

[1] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.

[2] J. Löfberg, "YALMIP: A toolbox for modeling and optimization in MATLAB," in *2004 IEEE international conference on robotics and automation (IEEE Cat. No. 04CH37508)*. IEEE, 2004, pp. 284–289.

[3] M. Grant, "Disciplined Convex Programming," Ph.D. dissertation, Stanford University, Dec. 2004, ph.D. dissertation. [Online]. Available: https://web.stanford.edu/~boyd/papers/pdf/mcg_thesis.pdf

[4] M. Grant, S. Boyd, and Y. Ye, "Disciplined convex programming," in *Global Optimization: From Theory to Implementation*, L. Liberti and N. Maculan, Eds. New York: Springer, 2006, pp. 155–210.

[5] M. Schaller, G. Banjac, S. Diamond, A. Agrawal, B. Stellato, and S. Boyd, "Embedded code generation with cvxpy," *IEEE Control Systems Letters*, vol. 6, pp. 2653–2658, 2022.

[6] R. Fourer and M. J. Harrison, "A modern approach to computer systems for linear programming," 1978.

[7] *IBM Mathematical Programming System Extended/370 (MPSX/370) Program Reference Manual*, 2nd ed., IBM Corporation, 1976.

[8] W. Orchard-Hays, "History of mathematical programming systems," *Annals of the History of Computing*, vol. 6, no. 3, pp. 296–312, 1984.

[9] I. Marceau and T. W. Mason, "A matrix-generator system for linear programming problems," Center for Advanced Computation, University of Illinois at Urbana-Champaign, Urbana, IL, Tech. Rep. CAC Document No. 4, 1971. [Online]. Available: https://hdl.handle.net/2142/32405

[10] *MaGen Reference Manual*, Haverly Systems, Inc., Denville, NJ, 1977.

[11] *GAMMA 3.4 Programmer Reference*, Sperry Univac Computer Systems, St. Paul, MN, 1977.

[12] *MIPS III DATAFORM User Manual*, Ketron, Inc., Arlington, VA, 1975.

[13] R. Fourer, "Modeling languages versus matrix generators for linear programming," *ACM Trans. Math. Softw.*, vol. 9, no. 2, pp. 143–183, Jun. 1983. [Online]. Available: https://doi.org/10.1145/357456.357457

[14] A. Brooke, D. Kendrick, and A. Meeraus, *GAMS: A User's Guide*. Redwood City, CA: The Scientific Press, 1988.

[15] J. Bisschop, *AIMMS optimization modeling*, 2006.

[16] R. Fourer, D. M. Gay, and B. W. Kernighan, "A modeling language for mathematical programming," *Management Science*, vol. 36, no. 5, pp. 519–554, 1990.

[17] I. Dunning, J. Huchette, and M. Lubin, "JuMP: A modeling language for mathematical optimization," *SIAM Review*, vol. 59, no. 2, pp. 295–320, 2017.

[18] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.

[19] I. CVX Research, "CVX: Matlab software for disciplined convex programming, version 2.0," https://cvxr.com/cvx, Aug. 2012.

[20] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd, "A rewriting system for convex optimization problems," *Journal of Control and Decision*, vol. 5, no. 1, pp. 42–60, 2018.

[21] M. Udell, K. Mohan, D. Zeng, J. Hong, S. Diamond, and S. Boyd, "Convex optimization in Julia," in *2014 first workshop for high performance technical computing in dynamic languages*. IEEE, 2014, pp. 18–28.

[22] A. Agrawal, S. Diamond, and S. Boyd, "Disciplined geometric programming," *Optimization Letters*, vol. 13, pp. 961–976, 2019.

[23] A. Agrawal and S. Boyd, "Disciplined quasiconvex programming," *Optimization Letters*, vol. 14, pp. 1643–1657, 2020.

[24] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and Z. Kolter, "Differentiable convex optimization layers," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[25] D. Cederberg, W. Zhang, P. Nobel, and S. Boyd, "Disciplined nonlinear programming," 2025.

[26] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.

[27] E. Chu, N. Parikh, A. Domahidi, and S. Boyd, "Code generation for embedded second-order cone programming," in *2013 European Control Conference (ECC)*, 2013, pp. 1547–1552.

[28] M. Lubin, O. Dowson, J. D. Garcia, J. Huchette, B. Legat, and J. P. Vielma, "Jump 1.0: Recent improvements to a modeling language for mathematical optimization," 2023. [Online]. Available: https://arxiv.org/abs/2206.03866

[29] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[30] L. Bottou, F. E. Curtis, and J. Nocedal, "Optimization methods for large-scale machine learning," *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018. [Online]. Available: https://epubs.siam.org/doi/10.1137/16M1080173

[31] Y. LeCun, "A theoretical framework for back-propagation," in *Proceedings of the 1988 Connectionist Models Summer School*, vol. 1. CMU, Pittsburgh, Pa, 1988, pp. 21–28.

[32] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2008.

[33] P. C. Chakravarti, "Truncation error in interpolation and numerical differentiation," *Numer. Math.*, vol. 3, no. 1, p. 279–284, Dec. 1961. [Online]. Available: https://doi.org/10.1007/BF01386029

[34] P. Wolfe, "Checking the calculation of gradients," *ACM Trans. Math. Softw.*, vol. 8, no. 4, p. 337–343, Dec. 1982. [Online]. Available: https://doi.org/10.1145/356012.356013

[35] Wolfram Research, Inc., "Mathematica, Version 14.3," 2026, champaign, IL. [Online]. Available: https://www.wolfram.com/mathematica

[36] Maplesoft, a division of Waterloo Maple Inc., "Maple," Waterloo, Ontario. [Online]. Available: https://www.maplesoft.com

[37] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig, B. E. Granger, R. P. Muller, F. Bonazzi, H. Gupta, S. Vats, F. Johansson, F. Pedregosa, M. J. Curry, A. R. Terrel, v. Roučka, A. Saboo, I. Fernando, S. Kulal, R. Cimrman, and A. Scopatz,

"Sympy: symbolic computing in python," *PeerJ Computer Science*, vol. 3, p. e103, Jan. 2017. [Online]. Available: https://doi.org/10.7717/peerj-cs.103

[38] S. Laue, "On the equivalence of forward mode automatic differentiation and symbolic differentiation," *ArXiv*, vol. abs/1904.02990, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:102350450

[39] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: a survey," *J. Mach. Learn. Res.*, vol. 18, no. 1, p. 5595–5637, Jan. 2017.

[40] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: http://github.com/jax-ml/jax

[41] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, *PyTorch: an imperative style, high-performance deep learning library.* Red Hook, NY, USA: Curran Associates Inc., 2019.

[42] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, "Tensorflow: a system for large-scale machine learning," in *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'16. USA: USENIX Association, 2016, p. 265–283.

[43] U. Naumann, *The Art of Differentiating Computer Programs: An Introduction to Algorithmic Differentiation*, ser. Software, Environments, and Tools. Philadelphia, PA: SIAM, 2011.

[44] P. Herholz, X. Tang, T. Schneider, S. Kamil, D. Panozzo, and O. Sorkine-Hornung, "Sparsity-specific code optimization using expression trees," *ACM Trans. Graph.*, vol. 41, no. 5, May 2022. [Online]. Available: https://doi.org/10.1145/3520484

[45] A. H. Gebremedhin, A. Tarafdar, A. Pothen, and A. Walther, "Efficient computation of sparse hessians using coloring and automatic differentiation," *INFORMS J.*

on Computing, vol. 21, no. 2, p. 209–223, Apr. 2009. [Online]. Available: https://doi.org/10.1287/ijoc.1080.0286

[46] A. H. Gebremedhin, F. Manne, and A. Pothen, "What color is your Jacobian? Graph coloring for computing derivatives," *SIAM Review*, vol. 47, no. 4, pp. 629–705, 2005.

[47] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.

[48] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2003.

[49] T. A. Davis, *Direct Methods for Sparse Linear Systems*. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2006. [Online]. Available: https://epubs.siam.org/doi/book/10.1137/1.9780898718881

[50] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[51] I. S. Duff, A. M. Erisman, and J. K. Reid, *Direct Methods for Sparse Matrices*. Oxford, England: Oxford University Press, 1986.

[52] P. E. Gill, W. Murray, M. A. Saunders, and M. H. Wright, "Sparse matrix methods in optimization," *SIAM Journal on Scientific and Statistical Computing*, vol. 5, no. 3, pp. 562–589, 1984.